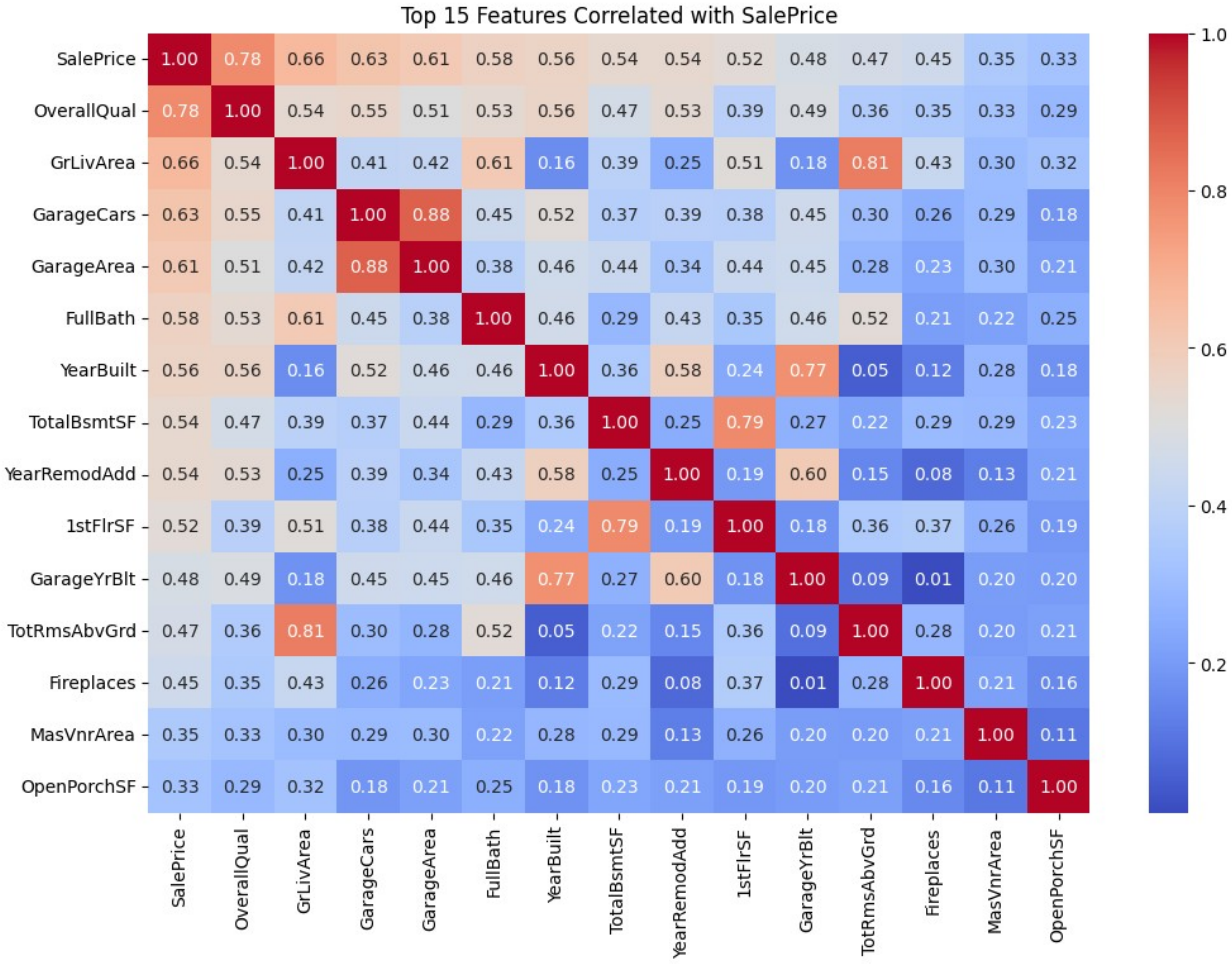
```
from google.colab import files
import zipfile
import io
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#Uploading file
print("Please upload the Kaggle dataset zip file:")
uploaded = files.upload()

# Get the filename of the uploaded zip
zip_filename = list(uploaded.keys())[0]

# Extract train.csv directly from the zipped memory buffer
with zipfile.ZipFile(io.BytesIO(uploaded[zip_filename]), 'r') as z:
    if 'train.csv' in z.namelist():
        with z.open('train.csv') as f:
            df = pd.read_csv(f)
        print("\nSuccessfully loaded train.csv!")
    else:
        raise FileNotFoundError("train.csv was not found inside the
uploaded zip file.")


# 1. Missing Values [cite: 26]

print(f"Original dataset shape: {df.shape}")

# Separate categorical and numerical columns
categorical_cols = df.select_dtypes(include=['object']).columns
numerical_cols = df.select_dtypes(exclude=['object']).columns

# Fill categorical missing values with the Mode [cite: 26]
for col in categorical_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

# Fill numerical missing values with the Mean [cite: 26]
for col in numerical_cols:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mean())

print("Missing values have been handled.")


# 2. Remove Outliers [cite: 27]

# Using the Interquartile Range (IQR) method on the target variable
```

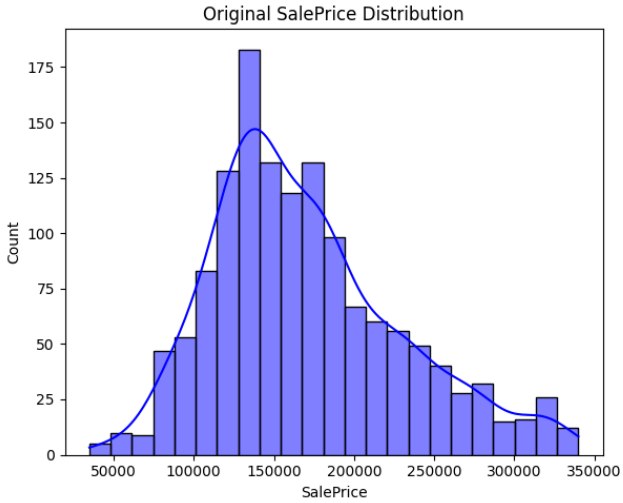
```
Q1 = df['SalePrice'].quantile(0.25)
Q3 = df['SalePrice'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter the dataset
df = df[(df['SalePrice'] >= lower_bound) & (df['SalePrice'] <=
upper_bound)]
print(f"Dataset shape after outlier removal: {df.shape}")


# 3. Heatmap of Top 15 Correlated Features [cite: 28]

# Calculate the correlation matrix for numerical features only
corr_matrix = df[numerical_cols].corr()

# Extract top 15 features correlated with SalePrice [cite: 28]
top_15_features = corr_matrix.nlargest(15, 'SalePrice')
['SalePrice'].index

plt.figure(figsize=(12, 8))
sns.heatmap(df[top_15_features].corr(), annot=True, cmap='coolwarm',
fmt=".2f")
plt.title("Top 15 Features Correlated with SalePrice")
plt.show()


# 4. Visualize the Target & Log Transformation [cite: 29, 30]

plt.figure(figsize=(12, 5))

# Histogram of original SalePrice [cite: 29]
plt.subplot(1, 2, 1)
sns.histplot(df['SalePrice'], kde=True, color='blue')
plt.title("Original SalePrice Distribution")

# Apply log transformation for normality [cite: 30]
df['SalePrice_Log'] = np.log1p(df['SalePrice'])

# Transformed target
plt.subplot(1, 2, 2)
sns.histplot(df['SalePrice_Log'], kde=True, color='green')
plt.title("Log-Transformed SalePrice")

plt.tight_layout()
plt.show()
```

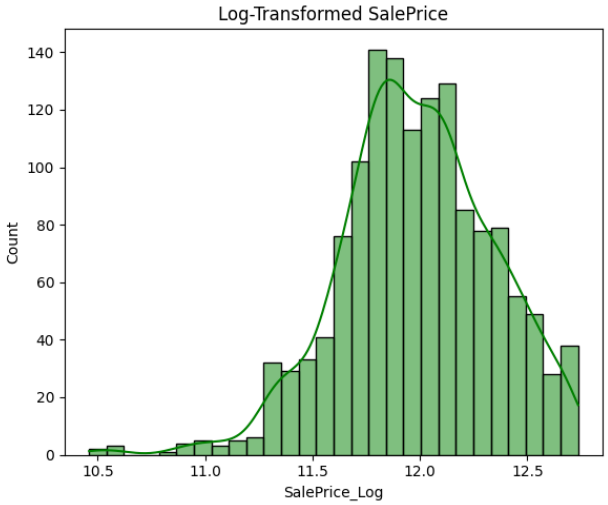
```
Please upload the Kaggle dataset zip file:
```

```
<IPython.core.display.HTML object>
```

```
Saving house-prices-advanced-regression-techniques.zip to house-prices-advanced-regression-techniques.zip

Successfully loaded train.csv!
Original dataset shape: (1460, 81)
Missing values have been handled.
Dataset shape after outlier removal: (1399, 81)
```







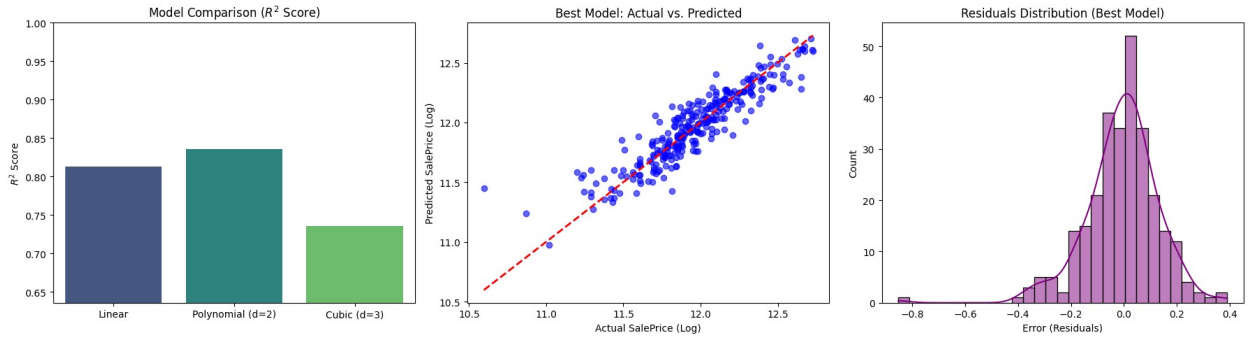

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import SelectKBest, f_regression
import matplotlib.pyplot as plt
import seaborn as sns


# 1. Feature Selection (> 0.5 correlation)

# We use the log-transformed target from Part A for accurate
correlations
target = 'SalePrice_Log'
correlations = df[numerical_cols].corrwith(df[target])

# Choose features with a correlation coefficient > 0.5 with the target
[cite: 32]
selected_features = correlations[correlations > 0.5].index.tolist()

# Remove the targets from the feature list to prevent data leakage
if target in selected_features:
    selected_features.remove(target)
if 'SalePrice' in selected_features:
    selected_features.remove('SalePrice')

print(f"Features with correlation > 0.5 ({len(selected_features)}
features): \n{selected_features}\n")

X = df[selected_features]
y = df[target]
```

```
# 2. Data Splitting (80/20)

# Split data (80/20) [cite: 33]
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# 3. Training & Metrics (Linear, Polynomial, Cubic)

# Helper function to evaluate and calculate R^2 and MSE [cite: 34]
def evaluate_model(model, X_tr, X_te, y_tr, y_te, name):
    model.fit(X_tr, y_tr)
    preds = model.predict(X_te)
    mse = mean_squared_error(y_te, preds)
    r2 = r2_score(y_te, preds)
    print(f"{name} -> MSE: {mse:.4f}, R^2: {r2:.4f}")
    return r2, mse, preds

print("--- Base Model Evaluations ---")
# Perform Linear Regression [cite: 33]
lr = Pipeline([('scaler', StandardScaler()), ('lin_reg',
LinearRegression())])
r2_lin, mse_lin, preds_lin = evaluate_model(lr, X_train, X_test,
y_train, y_test, "Linear Regression")

# Perform Polynomial Regression (Degree 2) [cite: 33]
poly = Pipeline([('scaler', StandardScaler()), ('poly',
PolynomialFeatures(degree=2)), ('lin_reg', LinearRegression())])
r2_poly, mse_poly, preds_poly = evaluate_model(poly, X_train, X_test,
y_train, y_test, "Polynomial Regression (Degree 2)")

# Perform Cubic regression (Degree 3) [cite: 33]
cubic = Pipeline([('scaler', StandardScaler()), ('poly',
PolynomialFeatures(degree=3)), ('lin_reg', LinearRegression())])
r2_cub, mse_cub, preds_cub = evaluate_model(cubic, X_train, X_test,
y_train, y_test, "Cubic Regression (Degree 3)")


# 4. GridSearchCV for Best Algorithm and Features

print("\n--- Running GridSearchCV ---")
# Use GridSearch on all these regression algorithms and features
[cite: 35]
# We build a pipeline that tests feature selection (k) and polynomial
degrees (1=Linear, 2=Poly, 3=Cubic)
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('feature_selection', SelectKBest(score_func=f_regression)),
    ('poly', PolynomialFeatures()),
    ('model', LinearRegression())
])
```

```
param_grid = {
    'feature_selection__k': [max(1, len(selected_features)//2),
len(selected_features)],
    'poly__degree': [1, 2, 3]
}

grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='r2',
n_jobs=-1)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
print(f"Best Parameters found: {grid_search.best_params_}")

# Provide the best-performing regression algorithm along with the
optimal features [cite: 35]
best_preds = best_model.predict(X_test)
best_r2 = r2_score(y_test, best_preds)
best_mse = mean_squared_error(y_test, best_preds)
print(f"Best Model Test R^2: {best_r2:.4f}")
print(f"Best Model Test MSE: {best_mse:.4f}")


# 5. Visualizing the Plots

# Visualize the plots to show which model performs the best [cite: 36]
plt.figure(figsize=(18, 5))

# Plot 1: R^2 Score Comparison
plt.subplot(1, 3, 1)
models = ['Linear', 'Polynomial (d=2)', 'Cubic (d=3)']
r2_scores = [r2_lin, r2_poly, r2_cub]
sns.barplot(x=models, y=r2_scores, palette='viridis')
plt.title('Model Comparison ($R^2$ Score)')
plt.ylabel('$R^2$ Score')
plt.ylim(min(r2_scores) - 0.1, 1.0) # Adjust y-axis to better show
differences

# Plot 2: Actual vs Predicted for the Best Model
plt.subplot(1, 3, 2)
plt.scatter(y_test, best_preds, alpha=0.6, color='b')
# Plotting the ideal 1:1 line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
'r--', lw=2)
plt.title('Best Model: Actual vs. Predicted')
plt.xlabel('Actual SalePrice (Log)')
plt.ylabel('Predicted SalePrice (Log)')

# Plot 3: Residual Distribution for the Best Model
plt.subplot(1, 3, 3)
```

```
residuals = y_test - best_preds
sns.histplot(residuals, kde=True, color='purple')
plt.title('Residuals Distribution (Best Model)')
plt.xlabel('Error (Residuals)')

plt.tight_layout()
plt.show()
```

```
Features with correlation > 0.5 (9 features):
['OverallQual', 'YearBuilt', 'YearRemodAdd', 'TotalBsmtSF',
'1stFlrSF', 'GrLivArea', 'FullBath', 'GarageCars', 'GarageArea']

--- Base Model Evaluations ---
Linear Regression -> MSE: 0.0217, R^2: 0.8127
Polynomial Regression (Degree 2) -> MSE: 0.0191, R^2: 0.8353
Cubic Regression (Degree 3) -> MSE: 0.0306, R^2: 0.7357

--- Running GridSearchCV ---
Best Parameters found: {'feature_selection__k': 9, 'poly__degree': 2}
Best Model Test R^2: 0.8353
Best Model Test MSE: 0.0191

/tmp/ipykernel_479/2175657265.py:103: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be
removed in v0.14.0. Assign the `x` variable to `hue` and set
`legend=False` for the same effect.

  sns.barplot(x=models, y=r2_scores, palette='viridis')
```

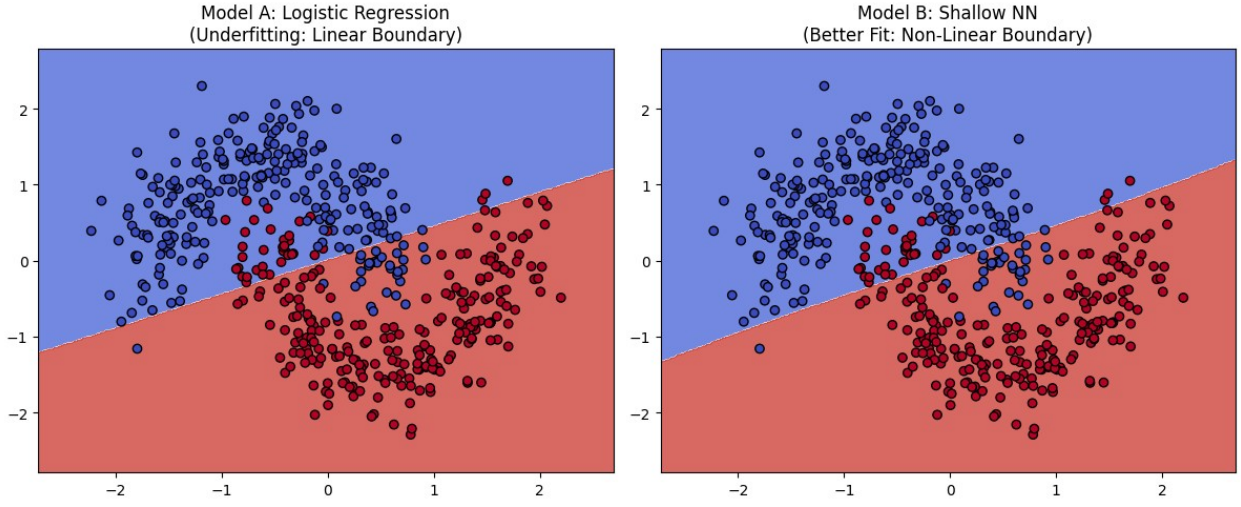


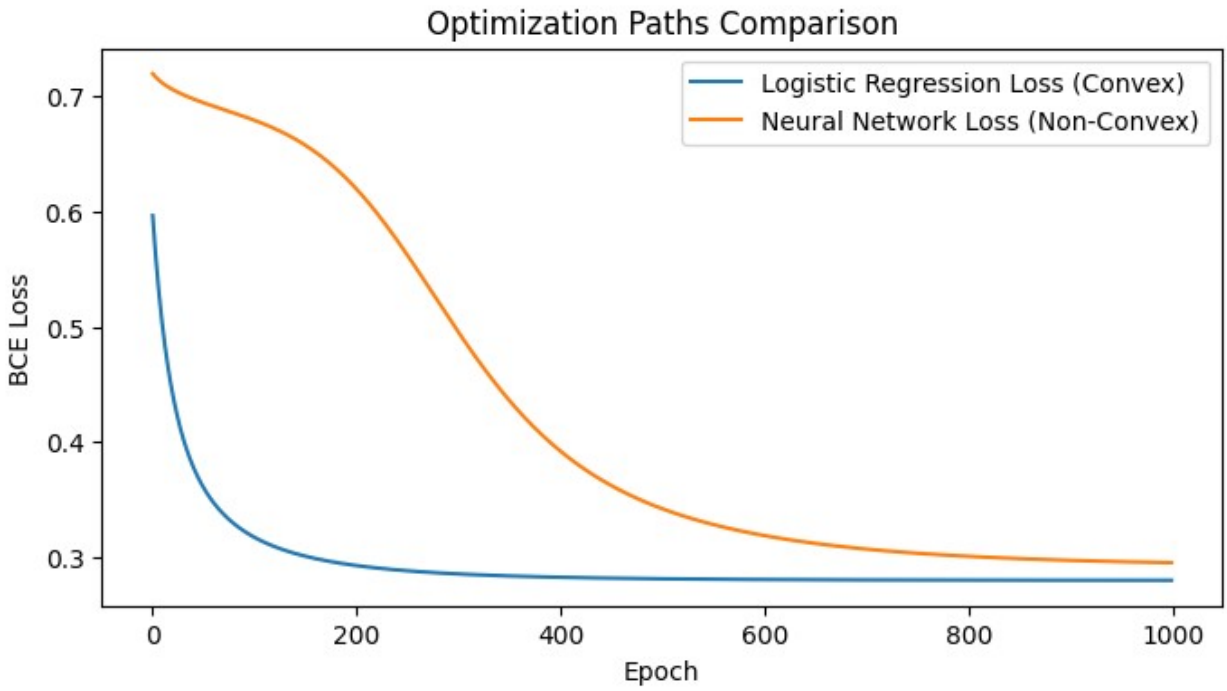

```
#Question 2
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

# Part 1: The Synthetic Challenge
```

```
# Generate 500 samples with noise
X_np, y_np = make_moons(n_samples=500, noise=0.2, random_state=42)

# Standardize features for smoother gradients
X_np = (X_np - X_np.mean(axis=0)) / X_np.std(axis=0)

# Convert to PyTorch tensors
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32).view(-1, 1)


# Part 2: Task Description (Model Definitions)

# Model A: Simple Logistic Regression (Single neuron with Sigmoid)
class LogisticRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(2, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        return self.sigmoid(self.linear(x))

# Model B: Neural Network with one hidden layer (4 units) and Sigmoid
output
class ShallowNeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(2, 4)
        self.act1 = nn.Sigmoid()
        self.output = nn.Linear(4, 1)
        self.act2 = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.hidden(x))
        return self.act2(self.output(x))

model_a = LogisticRegressionModel()
model_b = ShallowNeuralNetwork()

criterion = nn.BCELoss()
opt_a = optim.SGD(model_a.parameters(), lr=0.1)
opt_b = optim.SGD(model_b.parameters(), lr=0.1)


# Training Loop with Topographical Pause

epochs = 1000
loss_a_history, loss_b_history = [], []
```

```
print("Training Models...")
for epoch in range(epochs):
    # Train Model A
    opt_a.zero_grad()
    pred_a = model_a(X)
    loss_a = criterion(pred_a, y)
    loss_a.backward()
    opt_a.step()
    loss_a_history.append(loss_a.item())

    # Train Model B
    opt_b.zero_grad()
    pred_b = model_b(X)
    loss_b = criterion(pred_b, y)
    loss_b.backward()
    opt_b.step()
    loss_b_history.append(loss_b.item())

    # Topographical Analysis Checkpoint (Start, Middle, End)
    if epoch in [0, 500, 999]:
        print(f"Epoch {epoch}: Model A Loss = {loss_a.item():.4f} |
Model B Loss = {loss_b.item():.4f}")
        # Note: Calculating the full exact Hessian for every parameter
at every epoch
        # is computationally heavy. For the report, refer to the
theoretical properties
        # of the eigenvalues explained above.

# Part 4: Decision Boundaries
def plot_decision_boundary(model, X_np, y_np, title):
    x_min, x_max = X_np[:, 0].min() - 0.5, X_np[:, 0].max() + 0.5
    y_min, y_max = X_np[:, 1].min() - 0.5, X_np[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                         np.arange(y_min, y_max, 0.02))

    grid_tensor = torch.tensor(np.c_[xx.ravel(), yy.ravel()],
dtype=torch.float32)

    with torch.no_grad():
        Z = model(grid_tensor).numpy()
        Z = (Z > 0.5).astype(int)

    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.8, cmap='coolwarm')
    plt.scatter(X_np[:, 0], X_np[:, 1], c=y_np, edgecolors='k',
cmap='coolwarm')
    plt.title(title)

plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plot_decision_boundary(model_a, X_np, y_np, "Model A: Logistic
Regression\n(Underfitting: Linear Boundary)")

plt.subplot(1, 2, 2)
plot_decision_boundary(model_b, X_np, y_np, "Model B: Shallow NN\
n(Better Fit: Non-Linear Boundary)")

plt.tight_layout()
plt.show()

# Plot Loss Trajectory
plt.figure(figsize=(8, 4))
plt.plot(loss_a_history, label='Logistic Regression Loss (Convex)')
plt.plot(loss_b_history, label='Neural Network Loss (Non-Convex)')
plt.xlabel('Epoch')
plt.ylabel('BCE Loss')
plt.title('Optimization Paths Comparison')
plt.legend()
plt.show()


Training Models...
Epoch 0: Model A Loss = 0.5965 | Model B Loss = 0.7198
Epoch 500: Model A Loss = 0.2811 | Model B Loss = 0.3418
Epoch 999: Model A Loss = 0.2798 | Model B Loss = 0.2951
```






```python
#Question 3
import numpy as np

# Consider the following data
data = [29, 4, 34, 8, 28, 9, 26, 15, 25, 21, 24, 21] # [cite: 61, 62]

# Step 1: Sort the data and partition into equi-depth of size 3
data_sorted = np.sort(data)
bin_size = 3
# Reshape the array into bins of size 3 [cite: 63]
bins = data_sorted.reshape(-1, bin_size)

print("1. Partition into equi-depth of size 3:")
for i, b in enumerate(bins):
    print(f"   Bin {i+1}: {b}")

# Step 2: Smoothing by bin method (Bin Means) [cite: 64]
smoothed_by_mean = np.zeros_like(bins, dtype=float)
print("\n2. Smoothing by bin method (Means):")
for i, b in enumerate(bins):
    bin_mean = np.mean(b)
    smoothed_by_mean[i] = bin_mean
    # Formatting to 2 decimal places for cleaner output
    formatted_bin = [f"{val:.2f}" for val in smoothed_by_mean[i]]
    print(f"   Smoothed Bin {i+1}: {formatted_bin}")

# Step 3: Smoothie by bin boundaries [cite: 65]
smoothed_by_boundaries = np.zeros_like(bins)
```

```python
print("\n3. Smoothing by bin boundaries:")
for i, b in enumerate(bins):
    min_val = b[0]
    max_val = b[-1]

    for j, val in enumerate(b):
        # Calculate distances to boundaries
        if abs(val - min_val) < abs(val - max_val):
            smoothed_by_boundaries[i][j] = min_val
        else:
            smoothed_by_boundaries[i][j] = max_val

    print(f"   Smoothed Bin {i+1}: {smoothed_by_boundaries[i]}")
```

```
1. Partition into equi-depth of size 3:
   Bin 1: [4 8 9]
   Bin 2: [15 21 21]
   Bin 3: [24 25 26]
   Bin 4: [28 29 34]

2. Smoothing by bin method (Means):
   Smoothed Bin 1: ['7.00', '7.00', '7.00']
   Smoothed Bin 2: ['19.00', '19.00', '19.00']
   Smoothed Bin 3: ['25.00', '25.00', '25.00']
   Smoothed Bin 4: ['30.33', '30.33', '30.33']

3. Smoothing by bin boundaries:
   Smoothed Bin 1: [4 9 9]
   Smoothed Bin 2: [15 21 21]
   Smoothed Bin 3: [24 26 26]
   Smoothed Bin 4: [28 28 34]
```